

NFA To DFA Visual Simulator - Step By Step Code Explanation

1. PROJECT GOAL

This project is a separate NFA to DFA visual simulator. The user manually enters an NFA, then the program converts it into a DFA and prints clean transition tables.

The full path is: manual NFA input -> transition storage -> subset construction -> DFA table -> string testing.

2. MAIN.JAVA

Main.java contains the main method. It creates the UI object and starts the menu. The entry point stays very small so the rest of the logic is in focused classes.

3. UI.JAVA

UI.java controls the console experience. Option 1 lets the user enter a custom NFA. Option 2 displays the NFA table. Option 3 converts the NFA to a DFA. Option 4 displays the DFA table. Option 5 tests strings. Option 6 loads the professor demo NFA. For custom NFA input, the user enters states, alphabet, start state, final states, and transitions. Epsilon transitions can be written as eps, epsilon, or #.

4. STATE.JAVA

State.java represents one NFA state using a name such as q0. It stores transitions in a map from symbol to a set of states. A set is used because an NFA may move to multiple states on the same symbol.

The class also defines a special internal epsilon symbol. When printed, that symbol appears as eps.

5. NFA.JAVA

NFA.java stores all manually entered NFA data: states, alphabet, start state, final states, and transitions.

It validates important input rules, such as unknown states and symbols not found in the alphabet. It also has describe(), which prints the NFA table for presentation.

6. SUBSETCONSTRUCTION.JAVA

SubsetConstruction performs the standard NFA to DFA conversion. A DFA state is represented as a set of NFA states.

The first DFA state is epsilon-closure of the NFA start state. For each DFA state and symbol, the program computes move() and then epsilonClosure(). If the result is new, it becomes a new DFA state.

The empty set can become a DFA dead state when no NFA transition exists for a symbol.

7. DFASTATE.JAVA AND DFA.JAVA

DFAState.java stores the set of NFA states represented by one DFA state. It also records whether that DFA state is final. A DFA state is final if its NFA set contains at least one NFA final state.

DFA.java stores the DFA start state, final states, alphabet, and all transitions. Its describe() method prints the DFA state meanings and transition table.

8. TRANSITIONTABLEPRINTER.JAVA

TransitionTablePrinter is only responsible for display. It prints NFA and DFA transition tables in aligned columns. Start states are marked with -> and final states are marked with *.

Keeping this code separate makes the automata classes easier to understand, because display formatting is not mixed with conversion logic.

9. STRINGTESTER.JAVA

StringTester tests a string by starting from the DFA start state and following one transition per character.

If a character is not in the alphabet, the string is rejected. If the final reached state is final, the string is accepted. The class also prints the trace of states visited.

10. PROFESSOR DEMO

Menu option 6 loads a ready-made NFA. The language is all strings over {a,b} that end with ab.

Demo NFA states: q0, q1, q2. Alphabet: a, b. Start: q0. Final: q2. Transitions: q0 on a to q0 and q1; q0 on b to q0; q1 on b to q2.

Expected accepted strings: ab, aab, babab. Expected rejected strings: a, aba, bbb.

After the demo runs, the DFA remains loaded, so the user can choose option 5 and enter any custom test string.

11. HOW TO RUN

Command line: cd NFAToDFAVisualSimulator, then javac -d out src*.java, then java -cp out Main.

In IntelliJ: open the NFAToDFAVisualSimulator folder, mark src as Sources Root if needed, open Main.java, and click Run.

In NetBeans: create a Java project, copy the src files into its src folder, set Main as the main class, and run.